

My VizMachine instruction list:

LOAD1 180
LOAD1 281
LOAD1 382
LOAD1 483
ADD1 512
ADD1 634
XOR C56
JUMP C1E
LOAD2 A01
LOAD2 BFF
XOR 66B
ADD1 66A
ADD1 756
STORE 745
JUMP 020
STORE 545
HALT 000

Answers for the after-lab reflection questions:

- 1- The JUMP instruction's purpose is to make the pc (program counter) go to a new address, breaking the sequential flow and enabling conditional branching to different code blocks.
- 2- R0 provides a constant zero reference point essential for JUMP comparisons and initialization. Overwriting it corrupts all logic and JUMP.
- 3- XOR gives 0 only if inputs are equal. And this is the JUMP's working condition.
- 4- They are needed to ensure mutual exclusion. The first conditional JUMP skips the else block if true. The second unconditional JUMP skips the if part after the else block has finished running.
- 5- If removed, the if block would execute and then execute the else block immediately after, resulting in a false final calculation.
- 6- It splits C logic into the math part (ADD and XOR) and condition part (JUMP). This shows how high-level decisions are built from basic hardware commands.
- 7- No, HALT stops execution but does not reset registers. Registers maintain the final computed values from the last instruction executed.

Answers for the reflection questions:

- 1- I think the most difficult part is implementing the two's complement without a dedicated opcode and being forced to use XOR and ADD opcodes to calculate the negative version of a value.
- 2- This design deepens understanding by showing that high-level conditional logic is fundamentally constructed from low-level, binary operations and flow control at the hardware level.
- 3- It would need a positive feedback for a guess that turns out to be right, and a negative feedback for a wrong guess. This teaches the system to make better guesses next time.

CPU branching to intelligent decision-making systems:

CPU branching (using JUMP) is the most basic form of smart decision-making. When a computer checks a simple condition and chooses one path, it's making a decision. Modern CPUs use Branch Prediction, which is like a tiny, built-in learning model. It uses past success and failure records to guess where the code will go next. This cycle of guessing, checking the result, and learning from the mistakes is exactly AI systems work. In short, the simple JUMP instruction is the earliest ancestor of complex AI decisions.